KOGI

Independent Worker Operating System

Platform Architecture & Design — Part II

Market · Exchange · Idea & Design Studio · Community · Work & Operations

Document	Kogi Architecture & Design — Part II
Version	0.1.0 (Prototype)
Modules Covered	Market, Exchange, Studio, Community, Operations
Companion	Part I — Kernel, Portfolio, WBS, Crowdfunding, AO (see Part I)
Architecture Style	Modular services on Zig kernel · Go REST/gRPC · Scala analytics · Angular / JavaFX clients
Status	Active Design — Prototype Phase

PART 1

Market Module

The Kogi marketplace: listing, discovery, matching, and transaction infrastructure for independent workers, cooperatives, and communities

1.1 Overview & Purpose

The Market Module is Kogi's internal marketplace and discovery engine. It enables independent workers, cooperatives, and autonomous organizations to list services, products, skills, and resources; discover one another; and initiate work relationships — all within the Kogi ecosystem. Unlike external platforms, the Market is owned by its participants, governed by cooperative rules, and fully integrated with the Portfolio, WBS, Gig, and Crowdfunding systems.

Module ID	market
Service Type	REST API + Real-time search index + AI matching engine
Primary Actors	Independent workers, cooperatives, AOs, clients, employers, investors
Interfaces	Portfolio, Gig System, Crowdfunding, WBS, Community, Exchange
Storage	Listings index (full-text search) + Relational store (transactions, reviews)
AI Role	Semantic matching, demand forecasting, pricing suggestions, fraud signals

1.2 Market Segments

The Market is divided into six distinct segments, each with its own listing taxonomy, search facets, and transaction flows:

Segment	Description	Primary Listing Types	Transaction Model
Services Marketplace	Independent workers offer skills and services to clients	Gig, Contract, Retainer, Project-based	Gig story + escrow
Products & Goods	Physical and digital products sold by independent producers	Fixed price, Auction, Bundle	Order + fulfilment story
Capital & Investment	Investment opportunities, crowdfunding campaigns, revenue-share deals	Equity, Debt, Revenue-share, Grant	Crowdfunding + Capital Pool

Segment	Description	Primary Listing Types	Transaction Model
Resource Exchange	Equipment, space, tools, and assets available for rent or share	Hourly, Daily, Seasonal, Permanent	Exchange contract
Talent & Labor Pool	Searchable registry of worker skills and availability	Hourly, Project, Part-time, Full-time	Gig or Employment story
Cooperative & AO Network	Cooperative memberships, partner programs, community joining	Membership, Partnership, Contributor role	AO join + stake

1.3 Core Entities

Entity: Listing

Field	Type	Description
id	UUID	Unique listing identifier
type	ListingType	service | product | capital | resource | talent | cooperative
segment	MarketSegment	One of the six market segments
owner_id	UserID	Individual or AO that owns the listing
title	string	Short, searchable title (max 120 chars)
description	string	Rich text description with markdown support
tags	[]string	Searchable skill/category tags (max 20)
pricing	PricingModel	Fixed | Hourly | Auction | Negotiable | Free
price	Money?	Amount + currency (null if negotiable)
availability	Availability	Schedule, region, remote/on-site
media	[]Artifact	Photos, videos, portfolio samples
status	ListingStatus	draft | active | paused | sold | archived
metrics	ListingMetrics	Views, inquiries, conversions, rating
portfolio_ref	PortfolioItemID?	Link to owner's portfolio item
created_at	Timestamp	Creation time
updated_at	Timestamp	Last modification time

Entity: MarketInquiry

Field	Type	Description
id	UUID	Unique inquiry ID

listing_id	ListingID	Target listing
from_id	UserID	Inquiring party
message	string	Initial message
proposed_terms	Terms?	Budget, timeline, scope notes
status	InquiryStatus	pending | responded | accepted | declined | expired
gig_story_id	StoryID?	Linked Gig story once engaged
created_at	Timestamp	Inquiry timestamp

Entity: Review

Field	Type	Description
id	UUID	Review identifier
listing_id	ListingID	Reviewed listing
reviewer_id	UserID	Author of the review
engagement_id	StoryID	Completed gig/contract story being reviewed
rating	1–5	Numeric rating
dimensions	ReviewDimensions	Quality, communication, timeliness, value
body	string	Written review (max 1000 chars)
ai_sentiment	float	AI-scored sentiment (0.0–1.0)
verified	bool	Verified completed engagement
created_at	Timestamp	Review timestamp

1.4 AI Matching Engine

The Market's AI Matching Engine continuously surfaces the right opportunities to each user based on their portfolio context, skills, history, and current projects.

Signal	Source	Usage
Portfolio skills & history	User's Portfolio + WBS stories	Match talent listings and gig opportunities
Active project needs	Open Requirement stories in user's WBS	Surface relevant service and resource listings
Cooperative memberships	AO membership records	Preferential matching to co-op members
Past engagement quality	Review scores + completion rates	Rank listings by reliability and quality

Signal	Source	Usage
Budget signals	Portfolio capital resources	Filter listings to affordable range
Geographic & remote pref.	User profile + listing availability	Proximity and remote-work alignment
Community context	Community membership and activity	Surface listings from known, trusted peers
Crowdfunding momentum	Active campaign progress	Match investors to near-funded campaigns

Matching Algorithm

Three-pass ranking: (1) hard filter by segment + availability + budget → (2) semantic similarity score (embeddings on title+tags+description vs. user profile) → (3) trust re-rank by review quality + network proximity + AO membership overlap

1.5 Listing Lifecycle

1	**Draft** Owner creates a listing. Required fields validated. Media uploaded as Artifacts. AI Agent suggests tags, pricing benchmarks, and category from description.
2	**AI Review** AI Agent performs automated quality check: completeness score, spam/fraud signals, pricing sanity vs. market comps, duplicate detection. Flags surfaced to owner.
3	**Active** Listing published to the market index. Appears in search results and AI-generated opportunity feeds. Real-time view and inquiry metrics begin accumulating.
4	**Inquiry Received** Prospective buyer or client sends a MarketInquiry. Owner notified via Event Bus (market.inquiry.received). Message thread opens in Community comms.
5	**Engagement Initiated** Both parties accept terms. A Gig Story (or Contract story) is auto-created and linked to the listing. Escrow activated if applicable.
6	**Delivery & Review** Work completed. Review stories created for both parties. Ratings update listing's AI trust score. Listing metrics updated.
✓	**Complete / Renewal** Listing returns to Active for repeat engagements, or transitions to Sold/Archived. AI Agent suggests relisting price adjustments based on demand data.

1.6 Search & Discovery Architecture

Search Engine	Full-text + vector (embedding) hybrid search. Real-time index updates via Event Bus.
Query Types	Keyword, semantic (natural language), tag filter, geographic, availability calendar, price range
Facets	Segment, listing type, pricing model, rating, response time, AO membership, remote/on-site
Ranking	Relevance score × trust score × recency × personalization weight
Personalization	Per-user embedding derived from portfolio context; updated on each portfolio change event
Collections	Saved searches, curated collections, AI-generated "Opportunities for You" feed
Alerts	Saved search alerts fire market.listing.match event → Community notification

1.7 API Surfaces

Method	Endpoint	Description	Auth
GET	/market/listings	List + search listings with facets and pagination	Optional
POST	/market/listings	Create a new listing	Bearer JWT
GET	/market/listings/:id	Get full listing detail with metrics	Optional
PUT	/market/listings/:id	Update a listing	Owner
DELETE	/market/listings/:id	Archive a listing	Owner
POST	/market/listings/:id/inquire	Submit a market inquiry	Bearer JWT
GET	/market/listings/:id/reviews	List reviews for a listing	Optional
POST	/market/listings/:id/reviews	Submit a review (verified)	Bearer JWT
GET	/market/feed	Personalized AI opportunity feed	Bearer JWT
GET	/market/search	Full-text + semantic search	Optional
GET	/market/segments/:segment	Browse a specific market segment	Optional
GET	/market/analytics/listing/:id	Owner analytics: views, inquiries, conversions	Owner

📡 **Event Bus Topics**

- market.listing.created — new listing published to the index
- market.listing.updated — listing content or pricing changed
- market.inquiry.received — new inquiry submitted to a listing owner
- market.inquiry.accepted — both parties agreed; Gig story being created
- market.listing.match — saved search alert: new listing matches user criteria
- market.review.posted — review completed; trust score recalculated
- market.listing.sold — listing marked sold/completed

🔐 Security & Access

- Listings require authenticated owner — no anonymous listings
- AI fraud detection runs on every new listing and inquiry
- Review gating: reviews only permitted after a verified completed engagement story exists
- Cooperative listings: AO admin must approve listings made on behalf of the cooperative
- Price data treated as sensitive; AI pricing suggestions are suggestions only — never auto-applied

✅ Outcomes & Value

- Single discovery surface for all independent worker and cooperative market activity
- AI matching reduces time-to-engagement from days to minutes
- Verified reviews create a trust layer that external platforms cannot replicate for this community
- Market listings automatically linked to portfolio items — no context duplication
- Cooperative members preferentially matched to one another — network effects of trust

PART 2

Exchange Module

Peer-to-peer value transfer, resource sharing, escrow, revenue distribution, and cross-portfolio capital flows

2.1 Overview & Purpose

The Exchange Module is the value transfer layer of the Kogi platform. It handles all monetary and non-monetary value flows: payments between parties, escrow for gig engagements, revenue sharing from cooperatives and AOs, resource rental and barter, equity distributions, and cross-portfolio capital transfers. The Exchange sits between the Market (where agreements are initiated) and the Capital Pool system (where cooperative treasuries live), acting as the transaction and settlement engine.

Module ID	exchange
Service Type	REST API + Event-driven settlement processor + Ledger service
Primary Actors	Independent workers, clients, cooperative members, AO treasuries, investors
Interfaces	Market, Capital Pools, Crowdfunding, Portfolio, Gig System, Community
Storage	Double-entry ledger + Escrow vault + Distribution log + Transaction history
AI Role	Pricing suggestions, anomaly detection, distribution calculation, tax document generation

2.2 Exchange Transaction Types

Transaction Type	Description	Trigger	Settlement
Gig Payment	Client pays independent worker for completed gig story	Gig story status → done	Escrow release on confirmation
Subscription / Retainer	Recurring payment for ongoing service retainer	Scheduled billing cycle	Auto-debit on renewal date
Resource Rental	Payment for scheduled use of shared resource (equipment, space)	Reservation story confirmed	Pre-authorized hold; release on return
Product Sale	Payment for a physical or digital product listing	Order story created	Immediate capture; held pending shipment

Transaction Type	Description	Trigger	Settlement
Revenue Distribution	AO/Cooperative distributes earnings to members	Distribution story triggered by board/AI	Proportional split by stake or contribution
Equity Payout	Investment return distributed to stakeholders	Campaign milestone or exit event	Calculated by equity ledger; distributed to portfolios
Mutual Aid Transfer	Cooperative mutual aid fund disbursement to member	Application story approved by governance	Capital Pool → member account
Barter Exchange	Non-monetary exchange of services or resources	Barter agreement story created	Recorded as Credit/Debit in barter ledger
Tip / Gratuity	Voluntary additional payment to a worker	Manual trigger by client	Immediate transfer to worker wallet
Refund / Dispute	Return of funds following dispute resolution	Dispute story resolved	Reversal entry in ledger

2.3 Core Entities

Entity: ExchangeAccount

Field	Type	Description
id	UUID	Account identifier — one per user or AO
owner_id	UserID | AOID	Owning entity
account_type	AccountType	personal | cooperative | ao_treasury | escrow | investment
balance	Money	Current spendable balance
pending	Money	Holds: escrowed, pending settlement
currency	CurrencyCode	ISO 4217 currency (multi-currency supported)
ledger_id	LedgerID	Reference to double-entry ledger
status	AccountStatus	active | suspended | frozen | closed

Entity: Transaction

Field	Type	Description
id	UUID	Transaction identifier
type	TransactionType	See transaction types above
from_id	AccountID	Sending account
to_id	AccountID	Receiving account
amount	Money	Value transferred

fee	Money	Platform fee (if any)
story_ref	StoryID?	Linked Gig, Order, or Distribution story
escrow_id	EscrowID?	Active escrow vault reference
status	TxStatus	pending | processing | completed | failed | reversed
metadata	JSON	Arbitrary context: invoice number, tax fields, notes
created_at	Timestamp	Transaction initiation time
settled_at	Timestamp?	Actual settlement time

Entity: EscrowVault

Field	Type	Description
id	UUID	Escrow vault identifier
story_id	StoryID	Linked gig or contract story
payer_id	AccountID	Client holding funds in escrow
payee_id	AccountID	Worker awaiting release
amount	Money	Escrowed amount
conditions	[]EscrowCondition	Release conditions (story status, milestone, approval)
status	EscrowStatus	funded | released | disputed | refunded
dispute_id	DisputeID?	Active dispute case reference
auto_release_at	Timestamp?	Auto-release time if no dispute raised

Entity: DistributionRun

Field	Type	Description
id	UUID	Distribution identifier
ao_id	AOID	Issuing AO or cooperative
pool_id	CapitalPoolID	Source Capital Pool
total_amount	Money	Total to distribute
method	DistMethod	stake_weighted | contribution_weighted | equal | custom
recipients	[]DistRecipient	Member → amount pairs (AI-calculated)
status	DistStatus	draft | approved | processing | completed
approved_by	GovernanceVoteID?	AO vote approving distribution
executed_at	Timestamp?	Actual execution time

2.4 Ledger Architecture

The Exchange uses a double-entry accounting ledger as its source of truth. Every value transfer creates two journal entries — debit and credit — ensuring the ledger is always balanced and fully auditable.

Ledger Model	Double-entry — every transaction creates debit + credit entries
Account Types	Asset, Liability, Equity, Revenue, Expense — full COA per entity
Immutability	Ledger entries are append-only; corrections made via reversal entries only
Multi-currency	Transactions denominated in original currency; exchange rates snapshotted at settlement
Audit Trail	Every entry carries: creator ID, story ref, timestamp, digital signature
Reporting	AI Agent generates: P&L, cash flow, member statements, tax summaries from ledger
Cooperative Ledger	AOs have their own ledger namespace; member capital accounts tracked separately

2.5 Escrow Flow

1	**Gig Engagement Accepted** Market inquiry accepted → Gig story created. Exchange module creates an EscrowVault linked to the story. Client receives funding instructions.
2	**Escrow Funded** Client transfers agreed amount to EscrowVault. Transaction recorded as pending. Payee notified: funds secured. Work can begin.
3	**Work in Progress** Gig story progresses through WBS (in_progress → in_review). Escrow remains locked. Milestone releases configurable for long engagements.
4	**Delivery Confirmed** Client marks story as done OR auto-release timer expires (48h default after delivery). EscrowCondition evaluated.
5	**Escrow Released** Funds transfer from vault to worker's ExchangeAccount. Transaction completed. Ledger entries posted. Both parties prompted to leave reviews.
6	**Dispute Path (if raised)** Either party raises a Dispute story within the review window. Escrow frozen. AI Agent reviews evidence. Community mediator assigned if needed.
✓	**Dispute Resolution** Mediator decision posted to Dispute story. Funds released to prevailing party or split per ruling. Ledger corrected via reversal entries if needed.

2.6 Revenue Distribution Engine

The Distribution Engine handles cooperative and AO revenue sharing — from quarterly profit distributions to real-time gig revenue splits. It reads from the Capital Pool, applies the configured distribution method, and auto-calculates each member's share.

Distribution Method	Calculation	Use Case
Stake-weighted	member_share = (member_stake / total_stakes) × distributable_amount	Investment cooperatives, AOs with equity tiers
Contribution-weighted	member_share = (member_story_points / total_points) × amount	Open source communities, task-based co-ops
Equal (per capita)	member_share = total_amount / active_member_count	Flat cooperatives, mutual aid funds
Hours-weighted	member_share = (billable_hours / total_hours) × amount	Service cooperatives, consultant networks
Custom formula	Configurable expression per AO charter (e.g. 40% equal + 60% stake)	Complex multi-stakeholder AOs
Tiered	Different rates per membership tier (Founder / Builder / Member)	Crowdfunding-backed platforms

2.7 Resource Exchange & Barter

Beyond monetary transactions, the Exchange supports non-monetary value transfer: service bartering, cooperative resource lending, and time-banking (skill exchange by hours).

Barter Ledger	Separate ledger namespace for barter credits (non-convertible to cash by default)
Time Bank	Members earn time credits (1 hour = 1 credit) by helping others; spend credits to receive help
Resource Lending	Cooperative equipment loaned between members; usage tracked as rental transaction at $0
Contribution Credit	Open source and volunteer contributions tracked as non-monetary equity credits
Cross-Cooperative Barter	Two cooperatives can agree to exchange services; trade recorded bilaterally in both ledgers

2.8 API Surfaces

Method	Endpoint	Description	Auth
GET	/exchange/account	Get current user's exchange account and balance	Bearer JWT
POST	/exchange/transfer	Initiate a direct transfer between accounts	Bearer JWT
GET	/exchange/transactions	Paginated transaction history with filters	Bearer JWT
POST	/exchange/escrow	Create a new escrow vault for a gig story	Bearer JWT
PUT	/exchange/escrow/:id/release	Release escrow to payee	Payer or Auto
POST	/exchange/escrow/:id/dispute	Open a dispute on an escrow	Bearer JWT
POST	/exchange/distribute	Trigger a distribution run (AO admin)	AO Admin
GET	/exchange/distribute/:id	Get distribution run status and recipient details	AO Member
GET	/exchange/ledger	Read ledger entries (double-entry) for audit	Bearer JWT
POST	/exchange/barter/offer	Create a barter exchange offer	Bearer JWT
PUT	/exchange/barter/:id/accept	Accept a barter offer	Bearer JWT
GET	/exchange/statements/:period	Generate period financial statement	Bearer JWT

📡 Event Bus Topics

- exchange.escrow.funded — client has funded the escrow vault
- exchange.escrow.released — funds transferred to worker on story completion
- exchange.escrow.disputed — dispute opened; vault frozen pending resolution
- exchange.transfer.completed — direct transfer between accounts settled
- exchange.distribution.executed — AO revenue distribution processed for all recipients
- exchange.barter.agreed — barter exchange accepted by both parties
- exchange.account.low_balance — account balance below configured threshold

✅ Outcomes & Value

- Fully integrated escrow removes trust barrier for gig engagements — workers protected, clients protected
- Double-entry ledger provides audit-grade financial records for every cooperative and AO automatically

- Distribution engine handles the most complex multi-stakeholder splits automatically — zero manual calculation
- Barter and time-bank support enables non-capital workers to fully participate in the cooperative economy
- Dispute resolution built into the platform — no need for external arbitration services

PART 3

Idea & Design Studio

The Kogi creative pipeline: from raw spark to validated concept, prototype, and investable design artifact

3.1 Overview & Purpose

The Idea & Design Studio is the creative inception and development environment within Kogi. It provides a structured pipeline for independent workers, teams, cooperatives, and AOs to capture raw ideas; develop them into structured concepts; build and iterate prototypes; produce professional design artifacts; and ultimately graduate validated concepts into active Portfolio items, Market listings, or Crowdfunding campaigns. The Studio is deeply integrated with AI assistance at every stage — not to replace creativity, but to accelerate and structure it.

Module ID	studio
Service Type	REST API + Real-time collaborative workspace + AI pipeline orchestration
Primary Actors	Independent workers, design teams, cooperatives, AOs, investors (viewing)
Interfaces	Portfolio, Market, Crowdfunding, WBS, Exchange, Community
Storage	Versioned content store + Artifact storage + AI generation log
AI Role	Idea synthesis, concept structuring, market research, prototype generation, design critique

3.2 Studio Pipeline Stages

Every creative work in the Studio moves through six defined pipeline stages. Each stage produces a specific artifact and unlocks the next stage's tools.

Stage	Input	Output	AI Assistance	Graduate To
1. Spark	Raw thought, voice note, sketch, URL	Idea story — captured, tagged, searchable	Auto-tagging, similar idea detection, novelty score	Concept or Archive
2. Concept	Idea story + research prompt	Concept story — structured problem/solution,	Market research brief, competitive	Research or Prototype

Stage	Input	Output	AI Assistance	Graduate To
		audience, positioning	landscape, customer persona generation	
3. Research	Concept + research questions	Research story — validated assumptions, data, interviews, citations	Web research synthesis, interview guide generation, TAM estimation	Prototype or Business Case
4. Prototype	Concept + specs	Prototype story + design artifacts (wireframes, 3D specs, code stubs)	Wireframe generation, spec drafting, material/tech suggestions	Design or Validate
5. Design	Prototype feedback + constraints	Design story — polished artifact (brand, product, technical spec)	Design system generation, spec formatting, patent brief drafting	Validate or Launch
6. Validation	Design + go-to-market plan	Validated concept — ready for Portfolio, Market listing, or Crowdfunding	Financial model, pitch deck generation, risk assessment	Portfolio Item / Campaign / Market Listing

3.3 Core Entities

Entity: StudioItem

Field	Type	Description
id	UUID	Studio item identifier
stage	PipelineStage	spark | concept | research | prototype | design | validated
item_type	StudioType	idea | concept | research | prototype | design | business_case
title	string	Working title
description	RichText	Full description with markdown, embeds, references
owner_id	UserID | TeamID	Owning individual or team
collaborators	[]CollaboratorRef	Invited contributors with roles (viewer/editor/critic)
parent_id	StudioItemID?	Parent item in the pipeline (concept → research → prototype)
artifacts	[]Artifact	Attached files: sketches, specs, wireframes, models, code
ai_log	[]AIAction	Record of all AI assistance applied to this item
tags	[]string	Domain, technology, and category tags
market_potential	MarketPotential?	AI-estimated TAM, SAM, SOM
status	ItemStatus	active | paused | archived | graduated
portfolio_ref	PortfolioItemID?	Link once graduated to Portfolio
created_at	Timestamp	Creation time

updated_at	Timestamp	Last modified

Entity: Artifact

Field	Type	Description
id	UUID	Artifact identifier
studio_item_id	StudioItemID	Parent studio item
artifact_type	ArtifactType	sketch | wireframe | spec | model | code | document | video | audio | image | dataset
title	string	Artifact name
storage_url	string	Versioned object storage URL
version	SemVer	Version string (e.g. 1.0.0, 2.3.1)
mime_type	string	MIME type for rendering
ai_generated	bool	True if produced by AI generation tools
parent_version	UUID?	Previous version reference (lineage chain)
metadata	JSON	Domain-specific metadata (dimensions, tech stack, material, etc.)
created_at	Timestamp	Upload/generation time

Entity: AIStudioAction

Field	Type	Description
id	UUID	Action identifier
studio_item_id	StudioItemID	Target item
action_type	AIActionType	research | synthesize | generate | critique | suggest | format | estimate
prompt	string	User's instruction to the AI
model	string	Model used (e.g. claude-sonnet-4-5)
result_text	string?	Generated text output
result_artifact_id	ArtifactID?	Generated artifact (if applicable)
tokens_used	int	LLM tokens consumed (cost tracking)
user_rating	1–5?	User's quality rating of the AI output
created_at	Timestamp	Action timestamp

3.4 AI Capabilities by Stage

Stage	AI Tool	What It Does
Spark	Idea Classifier	Tags incoming idea, detects similarity to existing items, scores novelty 0–1
Spark	Idea Expander	Generates 5 related angle variations on a captured spark
Concept	Concept Structurer	Formats raw notes into: Problem, Solution, Target User, Key Differentiator, Initial Risks
Concept	Competitive Landscape	Web-searches competitors; returns structured comparison matrix
Concept	Persona Generator	Synthesizes 3 detailed customer personas from concept description and any interview notes
Research	Research Brief	Generates a structured set of research questions tailored to validate the concept
Research	Interview Guide	Creates a discovery interview script for target customer conversations
Research	TAM/SAM/SOM Estimator	Estimates market sizing from concept + available market data
Prototype	Spec Drafter	Converts concept description into a technical or product specification document
Prototype	Wireframe Generator	Produces wireframe descriptions or SVG sketches from UX flow descriptions
Prototype	Material/Tech Selector	Suggests optimal materials, tech stacks, or component choices for hardware/software prototypes
Design	Design System Generator	Creates a consistent design token set (colors, typography, spacing) from brand brief
Design	Patent Brief Drafter	Drafts a provisional patent application summary from technical spec
Validation	Financial Model	Generates 3-scenario (base/optimistic/downside) revenue model from concept + market data
Validation	Pitch Deck Outline	Structures a 10-slide pitch deck from validated concept data
Validation	Risk Matrix	Identifies top 10 risks with severity + mitigation suggestions

3.5 Collaboration Model

Studio items support real-time multi-person collaboration with role-based access. Collaborators can be individuals from outside the owner's workspace — invited via Community or Marketplace.

Role	Permissions	Typical Actor
Owner	Full control: edit, archive, graduate, invite, delete	Solo founder, team lead, cooperative project lead
Co-creator	Edit content, upload artifacts, run AI actions, comment	Team member, design partner, technical co-founder
Contributor	Comment, suggest changes (tracked), view all content	Advisor, community member, beta tester
Critic	Structured critique mode: rate and annotate artifacts only	Design reviewer, investor conducting DD, mentor
Viewer	Read-only: view all content and artifacts	Potential investor, partner, community member preview

3.6 Studio → Portfolio Graduation

A validated Studio item graduates into the main Portfolio system, becoming a fully operational business, product, or investment item. The graduation process is deliberate and creates clean linkage between the Studio's creative artifacts and the Portfolio's execution machinery.

1	**Validation Complete** Studio item reaches the Validated stage. Business Case, Financial Model, and Risk Matrix all present. Owner initiates graduation.
2	**Portfolio Item Created** A new Portfolio item is auto-created: type determined by item_type (product → Project, investment → Resource, cooperative concept → Program). Studio item linked as parent_ref.
3	**WBS Seeded** AI Agent generates an initial WBS from the Studio item's spec and business case: Work Packages → Themes → Epics → Stories pulled from the prototype's feature list and business model.
4	**Market Listing Drafted (optional)** Owner can simultaneously graduate to a Market listing. Listing pre-filled from Studio item title, description, media artifacts, and pricing model from financial model.
5	**Crowdfunding Campaign Drafted (optional)** If funding is needed, a Crowdfunding Campaign is pre-populated from the Business Case, Financial Model, and audience personas. Ready to review and launch.
✓	**Studio Item Archived** Original Studio item archived (not deleted). Full creative lineage preserved — from first Spark through all AI actions and artifact versions.

3.7 API Surfaces

Method	Endpoint	Description	Auth
GET	/studio/items	List all studio items for current user or team	Bearer JWT
POST	/studio/items	Create new studio item at any pipeline stage	Bearer JWT
GET	/studio/items/:id	Get full studio item with artifacts and AI log	Bearer JWT
PUT	/studio/items/:id	Update content, stage, status, or collaborators	Owner/Co-creator
DELETE	/studio/items/:id	Archive a studio item	Owner
POST	/studio/items/:id/artifacts	Upload or attach an artifact	Owner/Co-creator
GET	/studio/items/:id/artifacts/:aid	Download or view an artifact	Collaborator+
POST	/studio/items/:id/ai/:action_type	Invoke an AI Studio action	Owner/Co-creator
GET	/studio/items/:id/ai/history	Get full AI action log for an item	Owner
POST	/studio/items/:id/collaborate	Invite a collaborator with role	Owner
POST	/studio/items/:id/graduate	Graduate validated item to Portfolio	Owner
GET	/studio/feed	Discovery feed of public/shared studio items	Bearer JWT
POST	/studio/items/:id/critique	Submit a structured critique (Critic role)	Critic

📡 Event Bus Topics

- studio.item.created — new studio item entered the pipeline
- studio.item.stage_advanced — item moved to next pipeline stage
- studio.ai.action_completed — AI action finished; result available
- studio.artifact.uploaded — new artifact version attached to item
- studio.item.graduated — studio item promoted to Portfolio item
- studio.collaboration.invited — new collaborator added to a studio item
- studio.critique.submitted — structured critique posted by a Critic

✅ Outcomes & Value

- Creative ideas never lost — every spark captured, tagged, and searchable regardless of where it started

- AI assistance compresses research, spec-writing, and financial modeling from weeks to hours
- Pipeline structure ensures only validated, investment-ready concepts graduate to the Portfolio
- Full creative lineage preserved — investors and collaborators can trace an idea from spark to product
- Collaborative critic and review model brings external expertise without requiring full team onboarding
- Tight Market and Crowdfunding integration means a validated idea can launch to market in one session

PART 4

Community Module

Spaces, rooms, chat, social networking, and knowledge commons for independent workers, teams, cooperatives, and open source communities

4.1 Overview & Purpose

The Community Module is the social fabric of the Kogi platform. It provides the communication, discovery, and relationship infrastructure that connects independent workers with each other, with cooperatives, with clients, and with the broader independent work ecosystem. It combines real-time messaging, structured community spaces, a social knowledge graph, and cooperative-specific governance communication tools — all natively integrated with every other Kogi module.

Module ID	community
Service Type	REST API + WebSocket real-time + Graph store + Search index
Primary Actors	All Kogi users — individuals, teams, cooperatives, AOs, open source communities
Interfaces	All modules — Community is the communication layer across the entire platform
Storage	Message store (append-only) + Graph DB (social connections) + Presence service
AI Role	Content moderation, topic surfacing, knowledge synthesis, smart notifications, community health monitoring

4.2 Community Hierarchy

Community is organized in a four-level hierarchy. Every level has its own access controls, moderation tools, and communication features.

Level	Name	Description	Visibility	Size Range
1	Network	The top-level Kogi community — all users	Public to all Kogi users	Unlimited
2	Space	Domain or industry communities (e.g. "Web3 Builders",	Public, Invite-only, or Private	100–50,000

Level	Name	Description	Visibility	Size Range
		"Food Coops", "Indie Makers")		
3	Room	Focused discussion channels within a Space	Inherits Space visibility	5–5,000
4	Thread	Structured conversation within a Room (Q&A, decision, update, event)	Inherits Room visibility	2–unlimited replies

4.3 Space Types

Space Type	Purpose	Special Features
Open Community	Public topical community (e.g. "Freelance Designers")	Public join, SEO-indexed, open rooms
Cooperative Workspace	Private workspace for a specific cooperative or AO	Governance integration, member-only, treasury comms
Project Space	Dedicated comms space for a specific Portfolio project or WBS	Linked to stories and tasks, progress feed
Studio Collaboration	Creative team space for a Studio item pipeline	Artifact sharing, critique threads, AI action feed
Market Network	Community of practice in a Market segment (e.g. "Trades Contractors")	Listing notifications, referral threads, review discussions
Learning Hub	Structured knowledge and mentorship community	Courses, AMAs, office hours, resource library
Event Space	Time-bounded community for events, sprints, hackathons, or campaigns	Live sessions, RSVP system, post-event archive

4.4 Core Entities

Entity: Space

Field	Type	Description
id	UUID	Space identifier
name	string	Display name
slug	string	URL-friendly unique slug
space_type	SpaceType	See Space types above
owner_id	UserID | AOID	Owning individual, team, or AO

visibility	Visibility	public | invite_only | private
member_count	int	Current active member count
rooms	[]RoomRef	Child rooms in this space
linked_portfolio_ref	PortfolioItemID?	Linked portfolio item (for cooperative/project spaces)
cover_image	ArtifactID?	Space branding image
rules	[]string	Community guidelines
moderation	ModerationConfig	AI moderation settings, human moderator IDs
status	SpaceStatus	active | archived | suspended

Entity: Room

Field	Type	Description
id	UUID	Room identifier
space_id	SpaceID	Parent space
name	string	Room name (e.g. #general, #design-reviews, #governance)
room_type	RoomType	text | voice | video | announcement | governance | thread_board
purpose	string	One-line description of this room's focus
pinned_messages	[]MessageID	Pinned important messages
linked_stories	[]StoryID	WBS/portfolio stories surfacing updates here
ai_digest	DigestConfig	AI weekly/daily summary settings
archived	bool	Whether room is read-only archived

Entity: Message

Field	Type	Description
id	UUID	Message identifier
room_id	RoomID	Parent room
thread_id	ThreadID?	Parent thread (null = top-level)
author_id	UserID	Message author
content	RichText	Message body (markdown, embeds, @mentions, links)
attachments	[]Attachment	Files, images, studio items, portfolio refs, story cards
reactions	[]Reaction	Emoji reactions with reactor IDs
ai_moderation	ModerationResult?	AI content moderation result and confidence
edited_at	Timestamp?	Last edit time
deleted_at	Timestamp?	Soft delete time (content replaced with tombstone)

created_at	Timestamp	Post time

Entity: SocialConnection		
Field	**Type**	**Description**
id	UUID	Connection identifier
from_id	UserID	Initiating user
to_id	UserID | AOID	Target user or AO
connection_type	ConnType	follow | collaborator | teammate | mentor | client | partner
status	ConnStatus	pending | active | blocked
context	string?	How they connected (space, marketplace, studio collab)
created_at	Timestamp	Connection initiation time

4.5 Direct Messaging & Inbox

DM Types	1:1 direct message, Group thread (up to 25), Channel announcement
Rich Content	Markdown, code blocks, image embeds, voice messages, file attachments
Kogi Card Embeds	Any Kogi object (Story, Portfolio item, Listing, Studio item) can be shared as a rich card
Threading	Every DM thread can spawn sub-threads for specific topics
Search	Full-text search across all messages the user has access to
Notifications	Configurable per channel, thread, keyword, @mention, and connected-user activity
Encryption	End-to-end encrypted DMs (1:1 and group) — platform cannot read DM content
AI Inbox Assistant	Summarizes unread messages, surfaces action items from conversations, drafts replies

4.6 Social Graph & Profile

Every Kogi user has a social profile that is the public face of their independent worker identity. The profile integrates all modules — Market listings, Portfolio highlights, Studio showcase, cooperative memberships, and community contributions.

Profile Section	Content	Source Module
Identity	Name, avatar, tagline, location, availability status	User system
Skills & Expertise	Top skills, years of experience, endorsed capabilities	Portfolio + Reviews
Portfolio Highlights	Featured Portfolio items: products, projects, ventures	Portfolio
Studio Showcase	Public Studio items: designs, prototypes, concepts	Studio
Market Listings	Active service and product listings	Market
Cooperative Memberships	AOs and cooperatives the user belongs to	AO / Crowdfunding
Community Presence	Spaces joined, rooms active in, contributions made	Community
Work History	Completed gigs and engagements (with reviews)	Exchange + Market
Reputation Score	AI-computed trust score from reviews, completion rate, longevity	Market + Exchange
Open Availability	Calendar showing open capacity for new work	Scheduling

4.7 AI Community Features

Feature	Description	Trigger
Smart Notifications	AI filters notification stream to surface only high-signal events based on user context	Continuous background processing
Room Digest	Daily or weekly AI summary of each room's key discussions and decisions	Scheduled or on-demand
Knowledge Synthesis	AI aggregates answers from room history to surface prior art on a new question	New message containing a known question pattern
Content Moderation	AI scans messages for policy violations, spam, harassment, and low-quality content	Every message posted
Connection Suggestions	AI surfaces "people you should know" based on portfolio overlap, co-membership, project fit	Weekly for active users
Community Health Monitor	Tracks participation rates, new member onboarding friction, contributor burnout signals	Continuous; monthly report

Feature	Description	Trigger
Governance Comms	AI drafts governance proposals, vote summaries, and decision announcements for AOs	On governance event triggers
Trend Detection	AI surfaces emerging topics and skills gaining momentum in the network	Weekly cross-space analysis

4.8 Governance Communication Layer

For cooperatives and AOs, the Community module provides a dedicated Governance Room type with specialized tooling for democratic operations.

Governance Room	Dedicated room type linked to AO governance engine
Proposal Threads	Governance proposals surface as rich thread cards with: description, rationale, options, vote link
Vote Integration	Vote widget embedded in the room; members vote inline without leaving Community
Decision Archive	Every governance decision permanently archived in the room with full vote record
Town Halls	Scheduled voice/video sessions for cooperative all-hands — linked to agenda story in WBS
Delegate Messaging	Elected representatives can broadcast to members via Announcement room type
Dispute Mediation	Exchange disputes route to a private Mediation Room with mediator, both parties, AI log

4.9 API Surfaces

Method	Endpoint	Description	Auth
GET	/community/spaces	List discoverable spaces (public + member)	Optional
POST	/community/spaces	Create a new space	Bearer JWT
GET	/community/spaces/:id/rooms	List rooms in a space	Member+
POST	/community/spaces/:id/rooms	Create a room in a space	Space Admin
GET	/community/rooms/:id/messages	Get paginated message history	Member+
POST	/community/rooms/:id/messages	Post a message to a room	Member+

Method	Endpoint	Description	Auth
WS	/community/rooms/:id/live	WebSocket: real-time message stream	Member+
GET	/community/dm/:user_id	Get or create DM thread with user	Bearer JWT
POST	/community/dm/:user_id	Send a direct message	Bearer JWT
GET	/community/profile/:user_id	Get public user profile	Optional
POST	/community/connections	Send a connection request	Bearer JWT
GET	/community/feed	Personal activity feed	Bearer JWT
GET	/community/notifications	Paginated notification inbox	Bearer JWT
POST	/community/ai/digest/:room_id	Request AI room digest	Member+

📡 Event Bus Topics

- community.message.posted — new message in a room or DM
- community.space.created — new community space launched
- community.space.joined — user joined a space
- community.connection.requested — social connection request sent
- community.connection.accepted — connection established between two users
- community.mention — user mentioned in a message (@username)
- community.governance.proposal_posted — governance proposal published to space
- community.ai.digest_ready — AI room summary completed and available
- community.health.alert — AI detects community health issue (burnout, drop-off)

✅ Outcomes & Value

- Single communication layer across all Kogi modules — no context switching between Slack, Discord, and email
- Kogi Card embeds bring live portfolio, story, and listing data directly into conversations
- AI digest and synthesis makes large, active communities manageable for every member
- Social graph powers Market matching, Studio collaboration, and cooperative member discovery
- Governance communication built in — democratic cooperatives never need external voting tools
- End-to-end encrypted DMs ensure private conversations remain private even on a cooperative platform

PART 5

Work & Operations Module

Strategy, tactics, and operations for individuals, teams, cooperatives, and autonomous organizations

5.1 Overview & Purpose

The Work & Operations Module is the execution engine of the Kogi platform. It provides the full strategic and operational stack for independent workers, teams, cooperatives, and AOs to plan, execute, track, and improve their work — at any scale. It builds upon the core WBS, Story, and Portfolio systems from Part I and extends them with strategy-to-execution alignment tools, OKR-style objective management, tactical planning, sprint and iteration management, operations runbooks, and role-based work orchestration across all actor types.

Module ID	operations
Service Type	REST API + Real-time collaborative workspace + AI planning assistant
Primary Actors	Individuals, teams, cooperative members, AO contributors, project managers
Interfaces	Portfolio, WBS, Story System, Community, Exchange, Market, Analytics (Scala)
Storage	Relational store (objectives, plans, runbooks) + Time-series (metrics, velocity)
AI Role	Strategic planning, OKR generation, sprint planning, blocker resolution, ops automation, reporting

5.2 Work Contexts — Actor Types

The Operations module adapts its interface and tooling based on the work context of the actor. Four distinct contexts share a common underlying architecture but surface differently:

Work Context	Description	Key Tools	Scale
Personal	Solo individual managing their own goals, habits, side projects, career, and independent work	Goals, habits, personal roadmap, journal, daily plan	1 person

Work Context	Description	Key Tools	Scale
Team	A group of workers (employees, contractors, co-founders) collaborating on shared projects	Sprint board, backlog, team capacity, retrospectives, OKRs	2–100 people
Cooperative	A member-governed cooperative managing operations, services, and shared resources	Member roles, committee work, service delivery, compliance, governance tasks	5–5,000 members
Autonomous Organization (AO)	A fully distributed, self-governing org running projects, investment, and community at scale	Multi-team coordination, AI-delegated tasks, ecosystem governance, contributor rewards	10–100,000 contributors

5.3 Strategy Layer — Goals, Objectives & Roadmap

Every work context in Kogi can define and track strategic intent from the highest level (Mission) down through actionable Goals and OKRs. The AI Strategy Assistant helps translate vision into executable plans.

Level	Story Type	Description	Time Horizon
Vision	Vision story	The long-term aspirational purpose of the individual, team, or cooperative	3–10 years
Mission	Mission story	The current stated mission — what the entity does and for whom	1–3 years
Goal	Goal story	Specific, measurable desired outcome aligned to Mission	1 year
Objective	Objective story	Concrete 90-day target contributing to a Goal (OKR-style)	90 days
Key Result	Outcome story	Measurable indicator of Objective achievement	90 days
Initiative	Initiative story	A major body of work delivering Key Results	1–6 months
Tactic	Tactic story	Specific action or campaign executing an Initiative	Days–weeks
Operation	Operation story	Recurring operational process supporting tactics and delivery	Recurring

Strategic Alignment Check

> The AI Strategy Assistant continuously monitors alignment: does current sprint work trace back to active Objectives? Are active Objectives contributing to any Goal? Gaps surfaced as misalignment alerts. Workers always know why their task matters.

5.4 Tactical Planning — Sprints, Iterations & Campaigns

Sprint (Software/Product)	2-week iteration with velocity tracking, story points, burndown, retrospective
Campaign (Marketing/BD)	Time-bounded goal-driven push: launch, acquisition, fundraising drive
Service Cycle (Cooperative)	Recurring service delivery period: weekly cleaning schedule, quarterly harvest, monthly distribution
Contribution Window (AO/OSS)	Open source iteration: feature freeze, contributor sprint, release window
Personal Week Plan	Individual weekly prioritization: themed days, time blocks, habits, top 3 outcomes
Workshop / Event	Single-day or multi-day focused work session with structured agenda stories

5.5 Operations Layer — Runbooks & SOPs

The Operations layer manages recurring processes, standard operating procedures, and compliance workflows. Particularly important for cooperatives and AOs that need operational consistency without centralized management.

Entity: Runbook

Field	Type	Description
id	UUID	Runbook identifier
name	string	Process name (e.g. "Monthly Revenue Distribution", "New Member Onboarding")
context	WorkContext	personal | team | cooperative | ao
trigger	RunbookTrigger	manual | scheduled | event | story_status
trigger_config	JSON	Trigger details (cron expression, event type, status condition)
steps	[]RunbookStep	Ordered list of steps; each maps to a Story template or API action
owner_id	UserID | TeamID	Responsible owner
ai_automated	bool	Whether AI Agent can execute this fully autonomously
last_run_at	Timestamp?	Last execution time
run_count	int	Total number of executions

avg_duration_min	int	Average runtime in minutes (tracked over time)
tags	[]string	Category tags for search

Example Runbooks shipped with Kogi:

Runbook Name	Context	Trigger	Steps
New Member Onboarding	Cooperative	member.joined event	Provision account → assign role → create orientation tasks → send welcome comms → schedule check-in
Monthly Revenue Distribution	Cooperative / AO	1st of each month (scheduled)	Calculate period revenue → deduct expenses → compute member shares → create distribution run → notify members → execute
Sprint Planning	Team	Sprint start (scheduled)	Pull backlog → AI story point estimates → capacity check → team assignment → announce sprint → create burndown
Sprint Retrospective	Team	Sprint end (scheduled)	Gather what-went-well/delta → AI synthesis → action items → store in improvement backlog
Client Invoice & Follow-up	Individual	Gig story completed	Generate invoice artifact → send to client → schedule follow-up reminder → update cash flow story
Release Pipeline	AO / OSS	Tag pushed to repo	Feature freeze stories → regression test suite → documentation update → release notes draft → deploy → announcement
Governance Proposal Cycle	Cooperative / AO	Proposal story created	Draft review period → community discussion window → voting opens → vote closes → result announced → action item created
Quarterly OKR Review	All contexts	End of quarter (scheduled)	Pull all Objective stories → AI grades each → flag at-risk → generate retrospective report → set next quarter objectives

5.6 Role-Based Work Orchestration

In team, cooperative, and AO contexts, work is orchestrated across roles. Kogi's role system is flexible — roles can be permanent, elected, rotating, or AI-delegated.

Role Type	Description	Work Context	AI Delegation
Owner / Founder	Full control of workspace and portfolio	Personal, Team	N/A
Team Lead	Manages sprint, assigns stories, runs retrospectives	Team	Sprint planning, retrospective synthesis
Contributor / Member	Executes stories and tasks; self-directs backlog	All	Story selection, time estimation
Committee Member	Governs specific domain (finance, tech, member services)	Cooperative, AO	Agenda preparation, meeting notes
Elected Representative	Represents member interests in governance	Cooperative, AO	Governance proposal drafting
AI Executor Agent	Fully autonomous execution of AI-delegatable stories	AO, Open Source	Release management, reporting, distribution, moderation
Observer / Advisor	Read-only access; can comment and critique	All	N/A
Client / Partner	External collaborator with scoped project access	Team, Cooperative	N/A

5.7 Personal Work Context — Individual OS

For individual independent workers, the Operations module functions as a Personal Work OS — a daily system for managing energy, focus, priorities, and progress across every dimension of life and work.

Tool	Description	Story Types Used
Morning Brief	AI-generated daily brief: today's priorities, open stories, meetings, health habits, weather, market feed	Objective, Tactic, Task, Milestone
Weekly Plan	Structured weekly theme and time-block planning with top 3 outcomes per day	Goal, Tactic, Operation
Life Dashboard	Single-view of all active portfolios: business, personal, investments, health, learning	All story types across all portfolios
Focus Mode	Pomodoro + single-story focused execution view; timer integration; distraction blocking	Task, Tactic

Tool	Description	Story Types Used
Review Journal	Weekly and monthly written reflection; AI prompts; linked to goal progress	Outcome, Retrospective, Vision
Energy & Habit Tracker	Daily habit completion (health, learning, creative); streaks; AI pattern analysis	Operation (recurring)
Career Roadmap	Long-horizon professional development plan: skills to build, certifications, network targets	Goal, Initiative, Objective

5.8 Team Work Context — Collaborative Execution

Agile Board	Kanban + Scrum hybrid: Backlog → Ready → In Progress → In Review → Done. Drag-and-drop. Filter by assignee, type, priority.
Sprint Management	Sprint creation with capacity planning, story point totals, burndown chart, velocity tracking across sprints
Retrospectives	Structured What Went Well / What To Improve / Actions. AI synthesis and action tracking.
Team Capacity	Per-member availability vs. committed story points. Overload and underload surfaced proactively.
Dependency Tracking	Story-to-story dependencies visualized. Blockers auto-escalate to Team Lead with AI-suggested resolution.
Team OKRs	Shared Objective and Key Result stories. Progress aggregated from individual contributing stories.
Meeting Management	Meeting agenda = ordered list of story cards. Action items auto-create Task stories. AI generates minutes.
Performance Dashboard	Velocity trend, completion rate, quality metrics, team health score — AI-generated weekly.

5.9 Cooperative Work Context — Member-Governed Operations

Cooperatives have distinct operational needs: service delivery governance, committee work, member management, compliance, and democratic decision-making all coexist. Kogi's cooperative work context unifies these.

Operational Area	Tools	Key Story Types
Service Delivery	Service calendar, job assignment, quality tracking, client management	Gig, Operation, Performance, Review
Member Management	Onboarding runbook, role assignment, contribution tracking, status management	Milestone, Operation, Task

Operational Area	Tools	Key Story Types
Committee Work	Committee sub-workspaces, agenda management, decision tracking, action items	Strategy, Tactic, Governance, Decision
Financial Operations	Revenue tracking, distribution runs, budget stories, grant applications	Capital, Performance, Business Case, Report
Compliance & Audit	Compliance story templates, audit trail, governance documentation, annual reports	Audit, Report, Documentation, Review
Member Education	Learning paths, policy documents, skills development stories for member growth	Research, Documentation, Milestone
Strategic Planning	Annual plan development, scenario planning, member input sessions	Vision, Mission, Goal, Strategy, Initiative

5.10 AO Work Context — Distributed Self-Governance at Scale

Autonomous Organizations represent the most complex work context — thousands of contributors across geographies, no central management, AI-augmented coordination, and democratic governance over everything from technical decisions to treasury management.

AO Operation	Description	AI Assistance Level
Contributor Coordination	Global contributor matching to stories based on skills, availability, and interest	High — AI handles initial matching and assignment suggestions
Treasury Management	Multi-pool capital management, budget proposals, spend approvals, distribution runs	High — AI models scenarios; humans vote on significant decisions
Release Orchestration	Feature freeze, testing, docs, release notes, deployment, announcement coordination	Very High — AI Agent can run releases semi-autonomously with human approval gates
Governance Cycles	Proposal intake, deliberation period, vote, execution, retrospective	Medium — AI drafts proposals and summaries; voting fully human
Contributor Rewards	Point accumulation, tier assignment, periodic distribution calculation	Very High — fully automated per governance-approved formula
Community Moderation	Content review, spam, conduct enforcement across all community spaces	High — AI first pass; human moderators for complex cases
Strategic Direction	Annual vision setting, roadmap prioritization, ecosystem partnership decisions	Low — AI provides analysis and scenario modeling; community decides
Ecosystem Partnerships	Partner AO and cooperative relationship management	Medium — AI identifies and screens; governance approves

5.11 Core Entities

Entity: Objective

Field	Type	Description
id	UUID	Objective identifier
title	string	Clear, measurable objective statement
context_id	UserID | TeamID | AOID	Owning work context
parent_goal_id	StoryID?	Parent Goal story
key_results	[]KeyResult	Measurable outcomes (OKR-style)
time_horizon	DateRange	Start and target completion dates
status	ObjectiveStatus	on_track | at_risk | behind | achieved | cancelled
ai_grade	float?	AI-computed progress grade 0.0–1.0
contributing_story_ids	[]StoryID	Stories in WBS directly contributing to this objective
review_notes	[]ReviewNote	Quarterly review notes and retrospective entries

Entity: Runbook

Field	Type	Description
id	UUID	Runbook identifier
name	string	Process name
context	WorkContext	personal | team | cooperative | ao
trigger	RunbookTrigger	manual | scheduled | event | story_status
steps	[]RunbookStep	Step definitions with story templates and automation
ai_automated	bool	Can AI Agent execute fully autonomously?
last_run_at	Timestamp?	Last execution timestamp
avg_duration_min	int	Historical average runtime

Entity: WorkPlan

Field	Type	Description
id	UUID	Plan identifier
name	string	Plan name (e.g. "Q3 Team Plan", "2026 Cooperative Annual Plan")
context_id	UserID | TeamID | AOID	Owning context
plan_type	PlanType	sprint | campaign | weekly | quarterly | annual

objectives	[]ObjectiveRef	Objectives this plan is designed to achieve
story_ids	[]StoryID	All stories planned for this period
capacity_hours	float?	Total available person-hours for the period
committed_points	int?	Total story points committed
status	PlanStatus	draft | active | complete | cancelled
created_at	Timestamp	Plan creation time
period	DateRange	Start and end dates of the plan

5.12 AI Operations Assistant

Capability	Description	Work Context
Strategic Brief	Generates a concise strategic situation brief from current Portfolio, Objective, and OKR data	All
OKR Generation	Drafts a set of quarterly Objectives and Key Results aligned to existing Goal stories	All
Sprint Planning	Selects, sizes, and assigns stories for an upcoming sprint based on team capacity and backlog priority	Team, AO
Blocker Resolution	Analyzes blocked stories, identifies root cause, and suggests concrete unblocking actions	All
Retrospective Synthesis	Reads team input, synthesizes patterns, generates action items, and tracks improvement over time	Team, Cooperative
Runbook Execution	Triggers and executes approved runbooks with human approval gates at configured checkpoints	Cooperative, AO
Ops Anomaly Detection	Monitors KPIs and flags when operational metrics deviate from plan (velocity drop, quality decline, etc.)	All
Capacity Forecasting	Models future team capacity against planned work to surface over/under commitment early	Team, Cooperative
Annual Planning Support	Facilitates structured annual planning: reviews prior year, benchmarks, drafts next year objectives	All

5.13 API Surfaces

Method	Endpoint	Description	Auth
GET	/ops/objectives	List objectives for current context	Bearer JWT
POST	/ops/objectives	Create a new Objective with Key Results	Bearer JWT
GET	/ops/objectives/:id/progress	Get AI-graded progress for an Objective	Bearer JWT
POST	/ops/plans	Create a work plan (sprint, campaign, weekly)	Bearer JWT
GET	/ops/plans/:id	Get plan with stories, capacity, and progress	Bearer JWT
POST	/ops/plans/:id/ai/sprint-plan	AI-generate sprint story selection	Team Lead
GET	/ops/runbooks	List available runbooks for context	Bearer JWT
POST	/ops/runbooks	Create a new runbook	Admin/Lead
POST	/ops/runbooks/:id/run	Execute a runbook (manual trigger)	Authorized Role
GET	/ops/runbooks/:id/history	Get runbook execution history	Admin
GET	/ops/dashboard/personal	Personal daily brief and life dashboard	Bearer JWT
GET	/ops/dashboard/team/:id	Team performance dashboard	Team Member
GET	/ops/dashboard/ao/:id	AO operational health dashboard	AO Member
POST	/ops/ai/brief	Generate a strategic brief for current context	Bearer JWT
POST	/ops/ai/retrospective	Run AI retrospective synthesis from team input	Team Lead
GET	/ops/metrics/velocity	Sprint velocity trend for a team or AO	Bearer JWT

📡 Event Bus Topics

- ops.objective.created — new Objective established in a work context
- ops.objective.at_risk — AI grades Objective below threshold; alert generated
- ops.objective.achieved — Objective marked complete with all Key Results met
- ops.plan.activated — work plan transitions from draft to active
- ops.sprint.started — sprint begins; stories locked and burndown initiated
- ops.sprint.completed — sprint ends; retrospective story created
- ops.runbook.triggered — a runbook began execution (manual or automated)
- ops.runbook.completed — runbook finished all steps successfully

- ops.blocker.ai_resolved — AI suggested resolution accepted and implemented
- ops.capacity.overloaded — AI detects team is over-committed for the period
- ops.ao.contributor_matched — AI matched a contributor to an open AO story

✅ Outcomes & Value

- Single Operations module covers all four work contexts with the same underlying architecture — no tool sprawl
- Strategy-to-execution alignment maintained automatically — every task traces to an Objective and Goal
- AI Operations Assistant handles sprint planning, retrospectives, and runbook execution — reducing overhead for every team type
- Cooperative and AO operational complexity managed without centralized management layer
- Runbook library enables cooperatives to encode institutional knowledge into repeatable, automated processes
- Personal Work OS brings the same rigor to individual independent workers as enterprise teams get from dedicated PM tools

PART 6

Cross-Module Integration Architecture

How Market, Exchange, Studio, Community, and Operations connect to each other and to Part I modules

6.1 Integration Matrix

Every Kogi module exposes integration points to all others via the Event Bus and direct API composition. The table below summarises the primary integration flows between the five Part II modules and the Part I kernel.

From \ To	Market	Exchange	Studio	Community	Operations
Market	—	Inquiry accepted → Escrow created	Validated Studio item → Listing created	Listing match → Space notification	Open Requirement story → Listing surfaced
Exchange	Distribution → Market announcement	—	Studio graduation → Campaign funding unlocked	Distribution executed → Member comms	Gig payment → Story status auto-updated
Studio	Item graduated → Market listing	Validated item → Crowdfunding drafted	—	Item published → Space broadcast	Graduation → WBS auto-seeded
Community	Listing match alert → Room notification	Distribution notified in Coop room	Studio critique from Community invite	—	Blocker story → Team room alert
Operations	Ops brief surfaces Market feed	Distribution run triggered by Runbook	OKR gap → Studio research item created	Retro → Room discussion opened	—

6.2 Part I ↔ Part II Integration

Kernel	All Part II modules register as kernel modules. Event Bus wires inter-module communication. Provisioner creates Market, Exchange, Studio, Community, and Ops sub-accounts on user creation.
Portfolio	Market listings reference Portfolio items. Studio items graduate into Portfolio. Operations Objectives link to Portfolio Programs. Exchange accounts map to Portfolio Capital resources.

WBS + Story System	Operations Work Plans own WBS. Studio items auto-seed WBS on graduation. Market gigs create Gig stories. Community governance votes create Decision stories. Exchange distributions create Distribution stories.
Crowdfunding + Capital Pools	Exchange Distribution Engine reads Capital Pools. Studio validated items can pre-populate Crowdfunding campaigns. Market Capital segment lists active campaigns. Community spaces host campaign updates.
Autonomous Organizations (AO)	AO governance flows through Community Governance rooms. AO treasury managed via Exchange. AO contributor coordination via Operations. AO product listings on Market. AO concept development in Studio.

6.3 Event Bus Integration Topology

The Event Bus is the connective tissue of the entire platform. The diagram below shows the primary cross-module event flows for key platform workflows.

Workflow	Events Fired	Modules Involved
User Provisions	user.provisioned → market.profile.created → exchange.account.created → studio.workspace.created → community.profile.created → ops.context.created	Kernel, Market, Exchange, Studio, Community, Operations
Gig Lifecycle	market.inquiry.accepted → exchange.escrow.funded → ops.story.started → ops.story.completed → exchange.escrow.released → market.review.posted	Market, Exchange, Operations
Idea to Product	studio.item.created → studio.item.stage_advanced (×5) → studio.item.graduated → portfolio.item.created → ops.wbs.seeded → market.listing.created	Studio, Portfolio, Operations, Market
Cooperative Revenue Share	ops.runbook.triggered (distribution) → exchange.distribution.executed → community.message.posted (member notify) → portfolio.capital.updated	Operations, Exchange, Community, Portfolio
AO Feature Release	ops.runbook.triggered (release) → ops.sprint.completed → studio.artifact.uploaded (release notes) → community.message.posted (announcement) → market.listing.updated	Operations, Studio, Community, Market
New Member Join AO	community.space.joined → ops.runbook.triggered (onboarding) → exchange.account.credited	Community, Operations, Exchange

Workflow	Events Fired	Modules Involved
	(welcome) → ops.wbs.seeded (orientation) → community.dm.sent (welcome)	

6.4 Data Tenancy & Privacy Architecture

Tenancy Model	User-scoped: every user's Portfolio, Studio, Operations data is private by default. Cooperative/AO data scoped to membership.
Public Surface	Market listings, Community public spaces, Studio public items, User profiles — explicitly opted in
DM Privacy	End-to-end encrypted 1:1 and group DMs — server cannot read content
Cooperative Data	AO member data visible to other members per AO governance rules — not to platform by default
AI Training	User data never used for model training without explicit opt-in per Kogi's data governance policy
Data Export	Users and AOs can export all their data at any time (GDPR/CCPA compliant) via API
Right to Delete	User deletion removes PII from all modules; anonymized ledger entries retained for cooperative audit compliance

Appendix A — Part II Module Summary Reference

Module	ID	Primary Store	Key Entities	AI Role	Part I Dependencies
Market	market	Listings index + Relational	Listing, Inquiry, Review	Matching, pricing, fraud detection	Portfolio, Gig, Crowdfunding
Exchange	exchange	Double-entry ledger + Escrow vault	ExchangeAccount, Transaction, EscrowVault, DistributionRun	Anomaly detection, distribution math, tax docs	Capital Pools, Portfolio, Gig
Studio	studio	Versioned content store + Artifact store	StudioItem, Artifact, AIStudioAction	Research, spec drafting, wireframes, financial model	Portfolio, WBS, Crowdfunding
Community	community	Message store + Graph DB	Space, Room, Message, SocialConnection	Moderation, digest, suggestion, health monitor	All modules (comms layer)
Operations	operations	Relational + Time-series	Objective, Runbook, WorkPlan	Planning, retros, runbook exec, forecasting	Portfolio, WBS, Story, AO

Appendix B — Part II Technology Stack

Module	Language / Runtime	Framework	Data Store	Notes
Market	Go 1.22	Gin + gRPC	PostgreSQL (listings) + Typesense (full-text) + pgvector (semantic)	AI embeddings via Anthropic API
Exchange	Go 1.22 + Zig	Gin + Zig ledger core	PostgreSQL (double-entry ledger) + Redis (escrow locks)	Zig ledger lib linked via CGo FFI
Studio	Go 1.22 + Scala 2.13	Gin (API) + Akka Streams (AI pipeline)	PostgreSQL (metadata) + S3-compatible (artifacts) + Mongo (versions)	Scala handles AI action streaming

Module	Language / Runtime	Framework	Data Store	Notes
Community	Go 1.22	Gin + Gorilla WebSocket	PostgreSQL (messages) + Redis (presence) + Neo4j (social graph)	WebSocket for real-time; SSE fallback
Operations	Go 1.22 + Scala 2.13	Gin (API) + Akka Streams (analytics)	PostgreSQL (plans, runbooks) + TimescaleDB (metrics)	Scala analytics from Part I reused
AI Layer	—	Anthropic API (Claude claude-sonnet-4-5)	AI action log in PostgreSQL	Used by all modules; central AI client

Appendix C — Zig Module Extensions

The following Zig modules are added to the kernel for Part II, extending the Part I kernel foundation:

Zig Module	File	Purpose
market.zig	kernel/src/market.zig	Listing data structures, inquiry state machine, review schema
exchange.zig	kernel/src/exchange.zig	Ledger entry types, escrow vault, transaction state machine
ledger.zig	kernel/src/ledger.zig	Double-entry ledger core: account, journal entry, balance computation
studio.zig	kernel/src/studio.zig	Pipeline stage state machine, artifact metadata, AI action log
community.zig	kernel/src/community.zig	Space/Room/Message data structures, social graph edge types
operations.zig	kernel/src/operations.zig	Objective, KeyResult, Runbook, WorkPlan data structures